

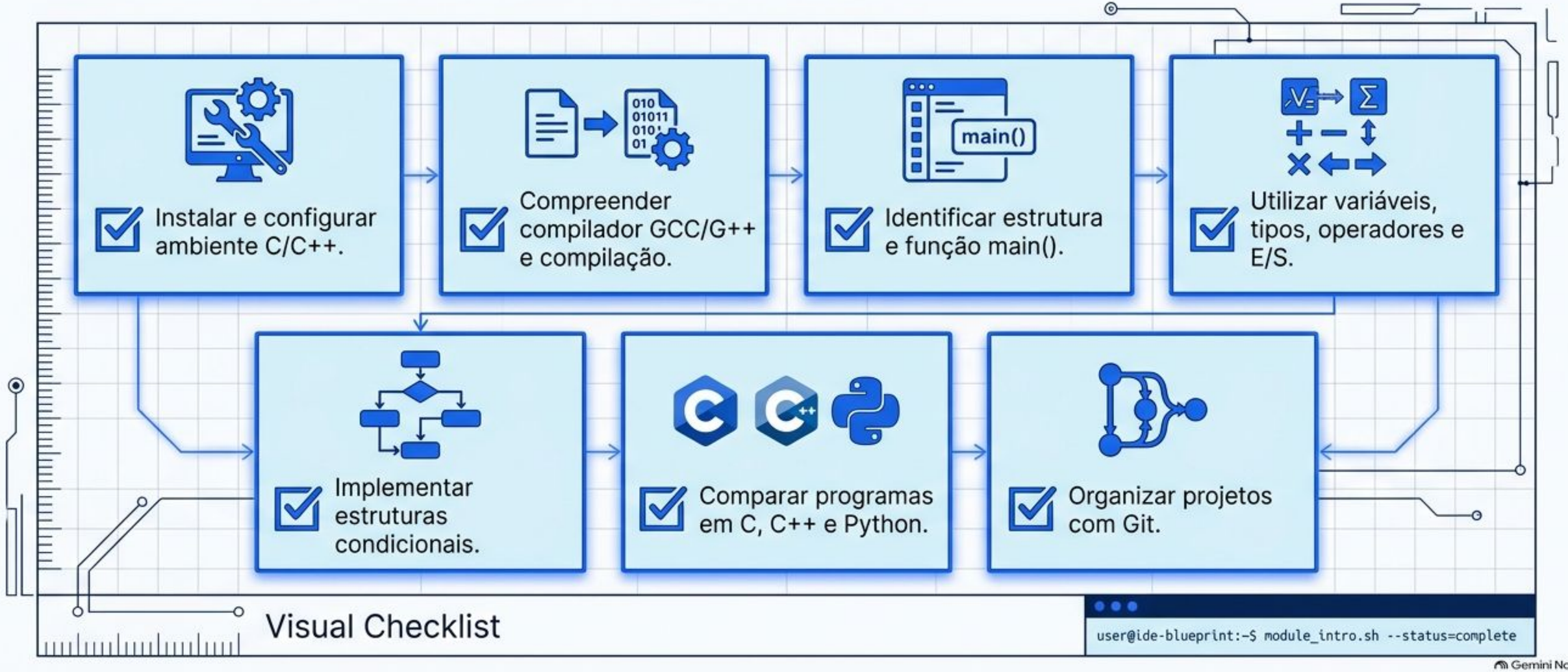
Integração de Linguagens: C, C++, Git e Projeto Final

Material Didático Completo — Do Ambiente de Desenvolvimento ao Controle de Versão

Gemini Notebook

1. Apresentação do Módulo e Objetivos

Percurso didático partindo da configuração, avançando para construção de programas e culminando em controle de versão (Git) e integração.



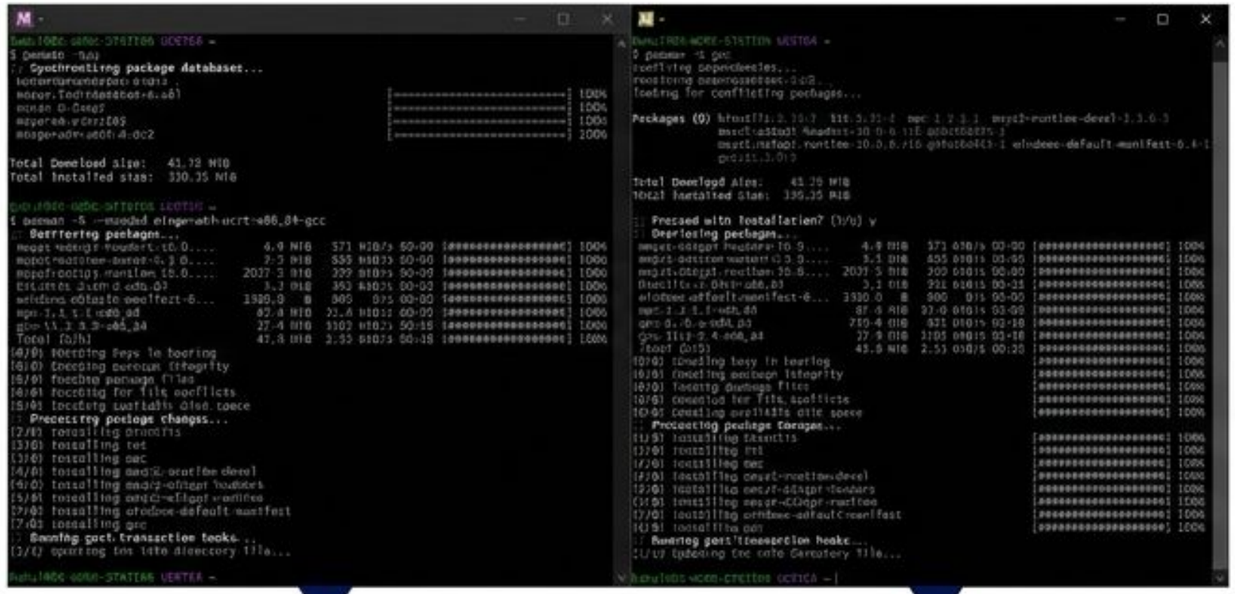
Gemini Notebook

2. Ferramentas do Ambiente e Configuração (MSYS2/GCC)

VS Code
(Editor)

MSYS2 & **GCC/G++**
(Compilação)

Git
(Controle de versão)



C:\msys64\ucrt64\bin



Etapa 1: MSYS2.

Etapa 2: GCC/G++.

Etapa 3: PATH.

Etapa 4: Verificação.

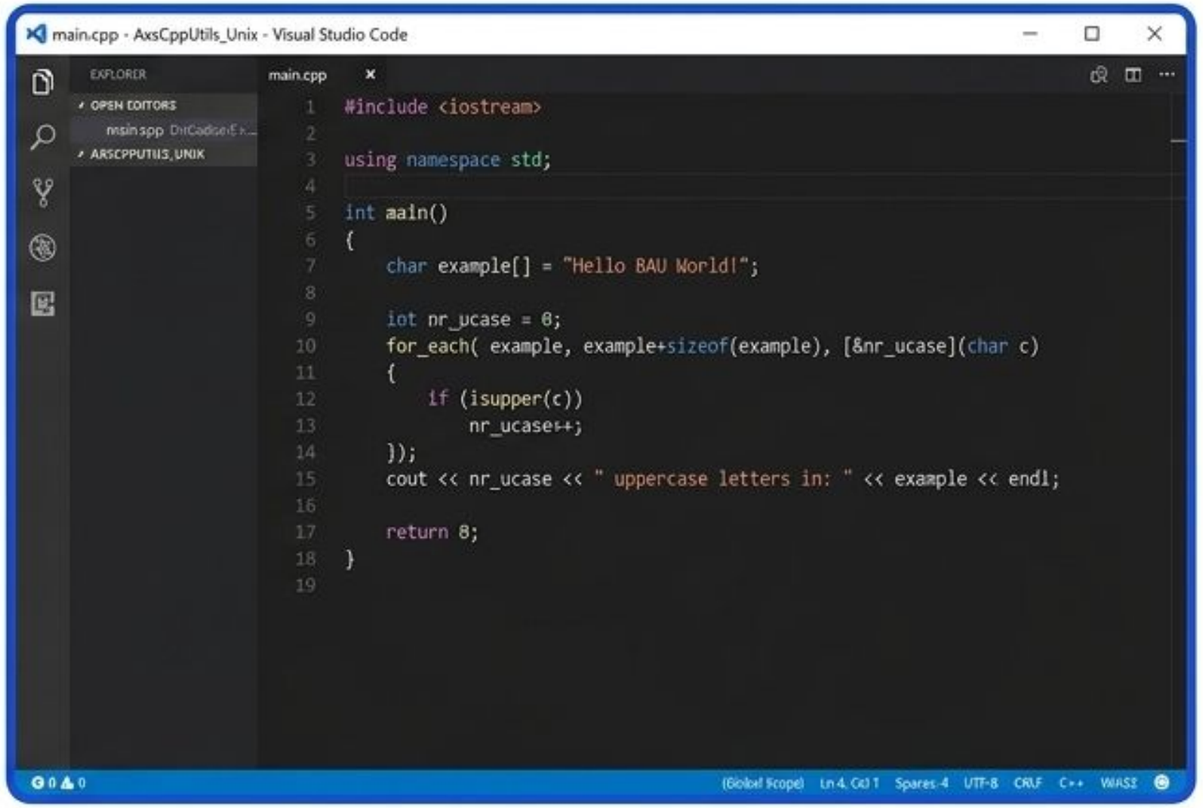
Gemini Notebook

Preparando o VS Code e Estrutura do Projeto

Extensions Checklist

- ✔ - C/C++ (Microsoft)
- ✔ - Python (Microsoft)
- ✔ - Extension Pack for Java (opcional)

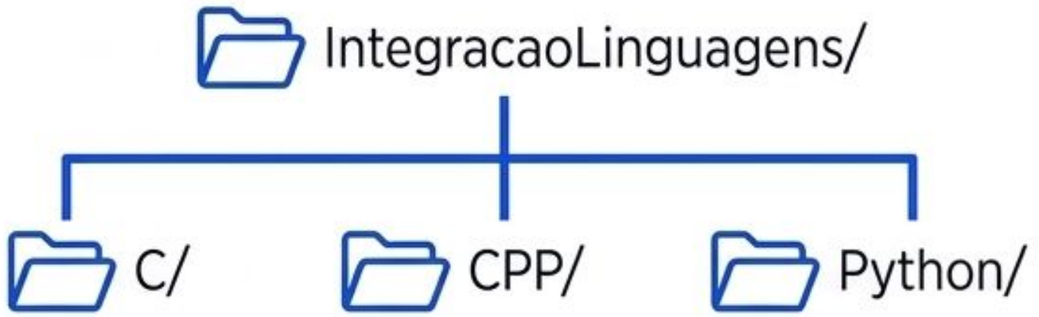
⚠ **A extensão C/C++ não substitui o compilador.**



CLI Window

```
> mkdir IntegracaoLinguagens
> cd IntegracaoLinguagens
> mkdir C mkdir CPP mkdir Python
> code .
```

Visual File Tree



Gemini Notebook

3. Introdução e Comparativo Inicial

Propósito geral, tipagem estática, baixo nível, memória/hardware. Usado em OS e embarcados.

Incorpora OOP, genérica e alto desempenho. Usado em jogos e simulações.

Diagnostic Matrix

Característica	C	C++	Python
Execução usual	Compilada	Compilada	Interpretada por uma implementação, normalmente CPython
Tipagem	Estática	Estática	Dinâmica
Declaração de variáveis	Tipo explícito	Tipo explícito	Geralmente sem declaração explícita do tipo
Ponto de entrada convencional	`main()`	`main()`	Código de módulo ou `if __name__ == '__main__':`
Entrada e saída	`printf, scanf`	`std::cout, std::cin`	`print, input`
Gerenciamento de memória	Manual ou por bibliotecas	Manual e recursos de abstração, como RAII	Gerenciamento automático de memória
Extensão típica	`.c`	`.cpp`	`.py`

Gemini Notebook

4. Estrutura do Programa e função main()

3-Pane IDE Layout

C/hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Ola, mundo!\n");
    return 0;
}
```

Usa `#include`, `main(void)`, `\n` e `return 0`.

CPP/hello.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Ola, mundo!"
<< '\n';
    return 0;
}
```

Usa `std::cout` e o prefixo `std::`

Python/hello.py

```python
def main():
    print("Ola, mundo!")

if __name__ == "__main__":
    main()
```

Usa a convenção `if __name__ == '__main__':`

```
Compilar e Executar
gcc C/hello.c -o C/hello.exe
.\C\hello.exe
g++ CPP/hello.cpp -o CPP/hello.exe
.\CPP\hello.exe
python Python/hello.py
Ola, mundo!
_
```

Gemini Notebook

5. **Variáveis, Tipos** e Atividade 1 (Cadastro)

Em C e C++, a declaração informa o espaço lógico e o tipo de dado.

Reference Table

Principais Tipos em C e C++

Tipo	Finalidade	Exemplo
`int`	Números inteiros	`int idade = 20;`
`float`	Números reais em precisão simples	`float altura = 1.75f;`
`double`	Números reais em precisão dupla	`double media = 8.5;`
`char`	Um caractere básico	`char letra = 'A';`
`bool`	Valores lógicos em C++	`bool aprovado = true;`

C/cadastro.c

```c
#include <stdio.h>
int main(void) {
    char inicial = 'A';
    int idade = 20;
    float nota = 8.5f;
    printf("Inicial: %c\n", inicial);
    printf("Idade: %d anos\n", idade);
    printf("Nota: %.1f\n", nota);
    return 0;
}
```

CPP/cadastro.cpp

```cpp
#include <iostream>
int main() {
    char inicial = 'A';
    int idade = 20;
    double nota = 8.5;
    std::cout << "Inicial: " << inicial << '\n';
    std::cout << "Idade: " << idade << " anos\n';
    std::cout << "Nota: " << nota << '\n';
    return 0;
}
```

Gemini Notebook

Atividade 1: Cadastro Simples (Python)

Python/cadastro.py

```python
inicial = "A"
idade = 20
nota = 8.5

print(f"Inicial: {inicial}")
print(f"Idade: {idade} anos")
print(f"Nota: {nota}")
```

Análise Didática

Em C, usa-se especificadores de formato %c, %d, %f com a função printf.

Em C++, utiliza-se o operador << com std::cout para combinar textos e valores.

Em Python, a tipagem é dinâmica e as usa-se f-strings para interpolação fluida de variáveis diretamente na string.

Gemini Notebook

6. Entrada e Saída: Atividade 2 (Calcular Idade)

C/idade.c

```c
#include <stdio.h>
int main(void) {
    int anoNascimento;
    int anoAtual = 2026;
    printf("Digite seu ano de nascimento: ");
    if (scanf("%d", &anoNascimento) != 1) {
        printf("Entrada invalida.\n");
        return 1;
    }
    if (anoNascimento < 1900 || anoNascimento > anoAtual) {
        printf("Ano de nascimento invalido.\n");
        return 1;
    }
    printf("Idade aproximada: %d anos\n", anoAtual - anoNascimento);
    return 0;
}
```

CPP/idade.cpp

```cpp
#include <iostream>
int main() {
    int anoNascimento;
    const int anoAtual = 2026;
    std::cout << "Digite seu ano de nascimento: ";
    if (!(std::cin >> anoNascimento)) {
        std::cout << "Entrada invalida.\n";
        return 1;
    }
    if (anoNascimento < 1900 || anoNascimento > anoAtual) {
      std::cout << "Ano de nascimento invalido.\n";
        return 1;
    }
    std::cout << "Idade aproximada: " << anoAtual - anoNascimento << " anos\n";
    return 0;
}
```

O operador **&** em C fornece o endereço de memória. O ano de referência fixado é 2026 para reprodutibilidade.

Gemini Notebook

Atividade 2: Calcular Idade (Python)

Python/idade.py

```
ano_atual = 2026
\033[36mtry:
    ano_nascimento = \033[36mt(input)("Digite
seu ano de nascimento: "))
    if 1900 <= ano_nascimento <= ano_atual:
        print(f"Idade aproximada: {ano_atual -
ano_nascimento} anos")
    else:
        print("Ano de nascimento invalido.")
\033[36mexcept ValueError:
    print("Digite um ano inteiro valido.")
```

Structural Note

Python converte a entrada explicitamente com int() e utiliza um bloco try/except robusto para lidar com erros de formatação de entrada em vez de checar retornos numéricos como em C/C++.

Gemini Notebook

7. Operadores e Atividade 3 (Calculadora)

Diagnostic Matrix

	Categoria	Operadores	Exemplo
1	Aritméticos	+, -, *, /, %	soma = a + b
2	Relacionais	==, !=, <, >, <=, >=	a >= b
3	Lógicos em C/C++	&&, ||, !	idade >= 18 && ativo
4	Atribuição	=, +=, -=, *=, /=	total += 5

C/calculadora.c

```c
#include <stdio.h>

int main(void) {
    double a, b;
    printf("Digite dois numeros: ");
    if (scanf("%lf %lf", &a, &b) != 2) return 1;
    printf("Soma: %.2f\n", a + b);
    printf("Subtracao: %.2f\n", a - b);
    printf("Multiplicacao: %.2f\n", a - b);
    printf("Multiplicacao: %.2f\n", a * b);
    if (b != 0.0) printf("Divisao: %.2f\n", a / b);
    else printf("Divisao por zero nao permitida.\n");
    return 0;
}
```

CPP/calculadora.cpp

```cpp
#include <iostream>
#include <iomanip>

int main() {
    double a, b;
    std::cout << "Digite dois numeros: ";
    if (!(std::cin >> a >> b)) return 1;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Soma: " << a + b << '\n';
    std::cout << "Subtracao: " << a - b << '\n';
    std::cout << "Multiplicacao: " << a * b << '\n';
    if (b != 0.0) std::cout << "Divisao: " << a / b << '\n';
    else std::cout << "Divisao por zero nao permitida.\n";
    return 0;
}
```

Gemini Notebook

Atividade 3: Calculadora (Python)

Python/calculadora.py

```python
try:
    a = float(input("Digite o primeiro numero: "))
    b = float(input("Digite o segundo numero: "))
    print(f"Soma: {a + b:.2f}")
    print(f"Subtracao: {a - b:.2f}")
    print(f"Multiplicacao: {a * b:.2f}")
    if b != 0:
        print(f"Divisao: {a / b:.2f}")
    else:
        print("Divisao por zero nao permitida.")
except ValueError:
    print("Digite numeros validos.")
```



Teste Orientado

Execute os três programas com entradas '10 e 4', e depois '10 e 0'.

Compare como cada linguagem e implementação trata a divisão por zero e a formatação de saída.

Gemini Notebook

8. Estruturas Condicionais (Verificar Aprovação)

Rule Note

Regra: Nota maior ou igual a 7,0 = aprovação. Inferior = recuperação.

C/aprovacao.c

```c
#include <stdio.h>

int main(void) {
    float nota;
    printf("Digite a nota do estudante: ");
    if (scanf("%f", &nota) != 1) return 1;
    if (nota < 0 || nota > 10) {
        printf("A nota deve estar entre 0 e 10.\n");
    } else if (nota >= 7.0f) {
        printf("Estudante aprovado!\n");
    } else {
        printf("Estudante em recuperacao.\n");
    }
    return 0;
}
```

CPP/aprovacao.cpp

```cpp
#include <iostream>

int main() {
    double nota;
    std::cout << "Digite a nota do estudante: ";
    if (!(std::cin >> nota)) return 1;
    if (nota < 0 || nota > 10) {
        std::cout << "A nota deve estar entre 0 e 10.\n";
    } else if (nota >= 7.0) {
        std::cout << "Estudante aprovado!\n";
    } else {
        std::cout << "Estudante em recuperacao.\n";
    }
    return 0;
}
```

Gemini Notebook

Condicionais (Python) e Desafio Prático

Python/aprovacao.py

```python
try:
    nota = float(input("Digite a nota do estudante: "))
    if not 0 <= nota <= 10:
        print("A nota deve estar entre 0 e 10.")
    elif nota >= 7:
        print("Estudante aprovado!")
    else:
        print("Estudante em recuperacao.")
except ValueError:
    print("Digite uma nota numerica valida.")
```

Desafio de Programação

Modifique os programas para 3 situações:

1. >= 7.0: Aprovado.
2. >= 4.0 e < 7.0: Recuperação.
3. < 4.0: Reprovado.

Regra: Inclua validação para impedir notas menores que zero ou maiores que dez.

Gemini Notebook

9. Laboratório de Atividades Codificadas

Activity Grid

1. Conversor de temperatura (C/C++)

In: `25°C` -> Out: `77°F.`

2. Média de três notas (C/C++/Py)

In: `7, 8, 9` -> Out: `8.00.`

3. Número par ou ímpar (C/C++)

In: `14` -> Out: `par.`

4. Maior entre dois números (C/C++/Py)

In: `12, 8` -> Out: `12 é o maior.`

5. Calculadora com menu (C/C++)

Menu + Tratar divisão por zero.

6. Classificação de idade (C/C++)

0-11, 12-17, 18+.

Critérios de Avaliação

Critério	Pontuação
Lógica e correção dos resultados	4,0
Uso adequado de tipos e operadores	2,0
Validação das entradas	1,5
Organização e legibilidade	1,5
Testes e comparação	1,0
Total	**10,0**

Gemini Notebook

10. Compilação, Avisos e Solução de Erros

Comandos Avançados

```
C: gcc -Wall -Wextra -std=c17 C/idade.c -o C/idade.exe
C++: g++ -Wall -Wextra -std=c++17 CPP/idade.cpp -o CPP/idade.exe

Python não exige compilação explícita neste fluxo.
```

Diagnostic Matrix (Erros Comuns)

Erro	Causa / Solução
`gcc is not recognized`	GCC ausente do Path (Verificar bin).
`undefined reference to main`	Função ausente ou erro na assinatura.
`expected ';'`	Ponto e vírgula ausente na instrução.
`No such file or directory`	Caminho ou nome do arquivo incorreto.
`Access is denied`	Executável em uso ou bloqueado pelo sistema.

Gemini Notebook

Experimento de Depuração

idade.c - Código C

```c
#include <stdio.h>
int main(void) {
    int idade = 20
    printf("Idade: %d\n", idade);
    return 0;
}
```

Missing ';'

Desafio: Identifique o erro, corrija, recompile. Em seguida, altere para `float` e observe os avisos.

Git Config Inicial

Terminal - Git Bash

```
git --version
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

Git registra o histórico. Vital para trabalho em equipe e desenvolvimento incremental.

Gemini Notebook

11. Git: Inicialização e .gitignore

CLI Window

Terminal – Git Inicialização

```
git init
git status
git add .
git commit -m "feat: adiciona exemplos
iniciais de C, C++ e Python"
git log --oneline
```

Sequência de comandos essenciais para iniciar, versionar e registrar o histórico do repositório.

IDE Panel

.gitignore

```
1  # Executáveis gerados
2  *.exe
3  *.out
4
5  # Arquivos temporários
6  *.tmp
7
8  # Cache Python
9  __pycache__/
10 *.pyc
11
12 # Configurações locais do editor
13 .vscode/
```

Configuração do arquivo para ignorar artefatos de build, caches e configurações pessoais, evitando o rastreamento desnecessário.

Gemini Notebook

Fluxo de Trabalho com Git e GitHub



CLI Window

Conexão Remota

```
git branch -M main
git remote add origin URL_DO_REPOSITORIO
git push -u origin main
```

Substitua a URL. Use branch de trabalho/Pull Request se houver regras de proteção na main.

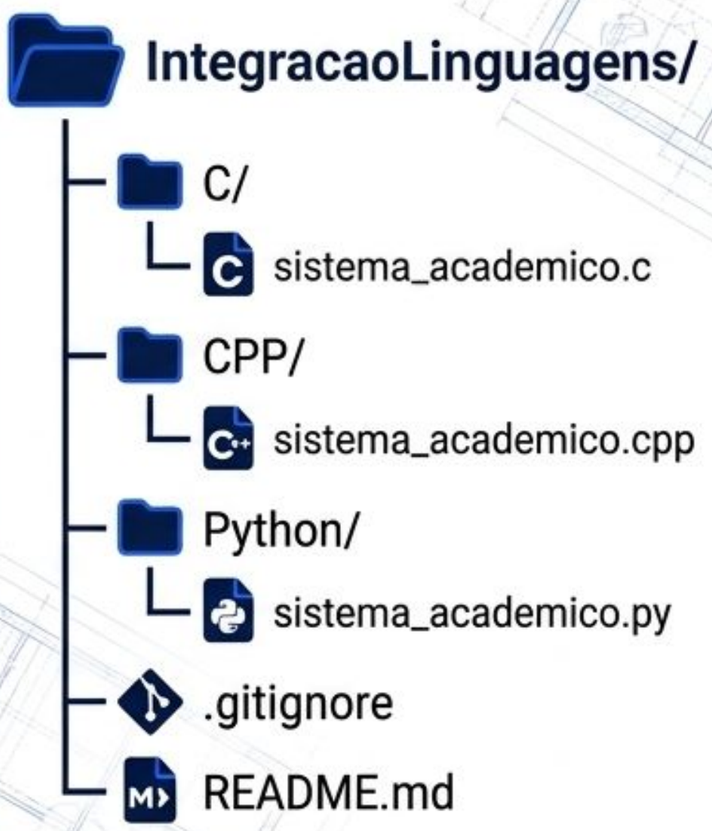
Gemini Notebook

12. Projeto Final Integrador

Requisitos Funcionais & Técnicos

- [x] Nome/matrícula/3 notas
  Entrada de dados do aluno.
- [x] Validar 0-10 e tratar erros
  Entrada e 0-10 e tratar erros.
- [x] Calcular média (2 casas)
  Calcular média (2 casas)
- [x] Classificar aprovação
  Classificar o com aprovação.
- [x] Implementar em C, C++, Py
  Implementar em C, C++, Py
- [x] **Versionar no Git e criar README.**
  Versionar no Git e criar README.

Estrutura de Repositório



Entregáveis e Relatório

1. **Código trilíngue**: Soluções funcionais em C, C++ e Python.
2. **Evidências de testes**: Prints ou arquivos de saída demonstrando cenários.
3. **Repo Git com histórico**: Histórico de commits estruturado.
4. **README**: Documentação clara e instruções de uso.
5. **Relatório comparativo**: Análise da experiência entre linguagens.

Perguntas

Quais exigiram mais declarações?

Diferença compilar vs executar?

Dificuldades da lógica nas 3 linguagens?

Contribuição do Git?

Gemini Notebook

Cronograma Sugerido e Referências

1. Setup VS Code, MSYS2, GCC.
2. Estrutura main() e compilação.
3. Variáveis e Tipos (Cadastro).
4. Entrada/Saída e Operadores (Calculadora).
5. Condicionais (Aprovação).
6. Comparação direta com Python.
7. Git, Commits e GitHub.
8. Projeto Integrador (Desenvolvimento/Apresentação).

Biblioteca e Referências

Livros

- The C Programming Language (Kernighan/Ritchie)
- The C++ Programming Language (Stroustrup)
- Pro Git

Documentações

- cppreference (C/C++)
- GCC oficial
- Python oficial em português

Orientação Pedagógica

O objetivo não é memorizar sintaxes, mas compreender algoritmos em diferentes linguagens e aplicar fundamentos de engenharia de software.

Gemini Notebook